test_first/06/app/controllers/projects_controller.rb
```
def create
  @workflow = CreatesProject.new(
    name: params[:project][:name],
    task_string: params[:project][:tasks])
  @workflow.create
  if @workflow.success?
    redirect_to projects_path
  else
    @project = @workflow.project
    render :new
  end
end
```

After you ask the workflow to create, check for success. If successful, redirect to the index URL as before. If not successful, take the project that was built but not saved, set it to the instance variable @project, and render the new action. By setting @project, any information that was entered into the form will still display when the form is rerendered.

What You've Done

You've written an entire (albeit small) piece of Rails functionality, starting with an end-to-end integration test and moving down to unit tests to make each part of the feature work. And it took only one chapter. It goes a lot faster when you don't stop to explain every line of code.

In the next few chapters you'll look at model testing, controller testing, and view testing, covering the libraries discussed in this chapter in more detail. You'll also learn about related topics, such as placing data in tests, testing for security, and testing JavaScript. After that you'll tackle some wider topics: how to test legacy code, how to keep your tests from becoming legacy code, how to test external services, and the like.

First, though, let's step back for a second and talk about what makes automated testing effective.